# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – ERROR ANALYSIS

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO2 – Manipulation | **Assessment:** | Assessment Tool 3 – ERROR ANALYSIS |
| **Weightage:** | 30% of CIA | | |
| **CO5 Statement:** | Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3) | | |
| **Student Name:** | Jayavarapu Raju | **Reg. No.:** | 192325117 |
| **Section / Batch:** | Slot-A | **Date:** | |

**Code of Conduct**

*I J.Raju(192325117)Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J.Raju

**Instructions**

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

| Section / Topic | Focus | Q Nos |
|---|---|---|
| **Inflectional Morphology and Derivational Morphology** | **Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.** | **Q1** |
| **Inflectional Morphology and Derivational Morphology** | **Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.** | **Q2** |
| **Finite-State Morphological Parsing** | **Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.** | **Q3** |
| **Finite-State Morphological Parsing** | **Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.** | **Q4** |
| **Porter Stemmer and Applications of Stemming in NLP** | **Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.** | **Q5** |
| **Porter Stemmer and Applications of Stemming in NLP** | **Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.** | **Q6** |
| **Porter Stemmer and Applications of Stemming in NLP** | **Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.** | **Q7** |

## Annexure A – Error Analysis

**Scenario-Based Error Analysis (1 – 4)**

**Question 1 – Morphological Analysis Error in Biomedical Dataset**

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

**Tasks**

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

**Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data**

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

**Tasks**

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

**Question 3 – Error Analysis of Stemming in News Article Classification**

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

**Tasks**

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

**Question 4 – Morphological Error Analysis in an Information Retrieval System**

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.

Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

**Tasks**

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

**Programming-Based Error Analysis (5 – 7)**

**Question 5 – Identify the Error, Rectify It, and Analyze the Output**

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

**Tasks**

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

**Question 6 – Error Analysis of a Finite-State Morphological Parser**

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
   print(parser(w))
```

**Tasks**

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

**Question 7 – Error Analysis of Morphological Feature Extraction**

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
```

```
stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

**Tasks**

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Error Identification | 20% | Accurately identifies all errors with clear understanding of issues | Identifies most errors with minor omissions | Identifies limited errors; some are missed | Fails to identify key errors |
| Root Cause Analysis | 30% | Clearly explains underlying causes with strong technical reasoning | Explains causes with moderate clarity | Partial explanation; weak reasoning | Unable to identify causes of errors |
| Correction & Justification | 25% | Provides correct solutions with strong justification and logic | Provides correct corrections with some justification | Corrections are partially correct or weakly justified | Incorrect or no corrections provided |
| Application of Concepts | 15% | Effectively applies relevant concepts to analyze and resolve errors | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply concepts |
| Clarity & Presentation | 10% | Presents analysis clearly, logically, and systematically | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |

| Q. No. / Criteria Level | Error Identification | Root Cause Analysis | Correction & Justification | Applications of Concepts | Clarity & Presentation | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |
| 4 | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# Question 1 – Morphological Analysis Error in Biomedical Dataset

## 1. Aim

To investigate how Porter Stemming affects biomedical words such as:

- infection
- infectious
- infected
- infect

and determine whether stemming removes important **derivational morphological information**.

### Important observation

Porter Stemmer produces approximately:

| Word | Porter Stem |
|---|---|
| infect | infect |
| infected | infect |
| infection | infect |
| infectious | infecti |

The important problem is:

`infection → infect` and `infectious → infecti`

The suffixes `-ion` and `-ious` are derivational suffixes and can change the grammatical category or semantic meaning of the word.

---

## 2. Preprocessing steps

The preprocessing pipeline is:

Biomedical Dataset
↓
Lowercase
↓
Remove punctuation
↓

Tokenization
↓
Remove unnecessary tokens
↓
Porter Stemming
↓
Compare original word with stem
↓
Identify morphological errors

---

# 3. Complete Python program

This version can work with a CSV file containing a `text` or `Text` column.

```
import pandas as pd
import re
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import nltk

# Download tokenizer resources
nltk.download("punkt")
nltk.download("punkt_tab")

# ----------------------------
# Load dataset
# ----------------------------

FILE = "PubMed20k.csv"

data = pd.read_csv(FILE)

# Find text column automatically
possible_columns = ["text", "Text", "abstract", "Abstract", "sentence"]

text_column = None

for col in possible_columns:
    if col in data.columns:
        text_column = col
        break

if text_column is None:
    raise ValueError(
        "No text column found. Rename your dataset text column to 'text'."
    )

# ----------------------------
# Preprocessing function
# ----------------------------

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z\s]", " ", text)
```

```
    tokens = word_tokenize(text)

    tokens = [
        word for word in tokens
        if word.isalpha()
    ]

    return tokens


# ------------------------------
# Porter Stemmer
# ------------------------------

stemmer = PorterStemmer()

records = []

for text in data[text_column].dropna().head(1000):

    tokens = preprocess(text)

    for word in tokens:
        stem = stemmer.stem(word)

        records.append({
            "Original": word,
            "Stem": stem
        })


result = pd.DataFrame(records)

# Remove duplicate word-stem pairs
result = result.drop_duplicates()

print("\nOriginal words and Porter stems:")
print(result.head(30))

# Save results
result.to_csv("biomedical_stemming_results.csv", index=False)

# ------------------------------
# Investigate important words
# ------------------------------

target_words = [
    "infect",
    "infected",
    "infection",
    "infectious"
]

print("\nTarget word analysis:")
```

```
for word in target_words:
    stem = stemmer.stem(word)
    print(f"{word:15} -> {stem}")
```

---

# 4. Expected output

Target word analysis:

```
infect          -> infect
infected        -> infect
infection       -> infect
infectious      -> infecti
```

---

# 5. Identify derivational errors

infection → infect
infectious → infecti

## Why is this a problem?

Consider:

infect
infection
infectious

They are related, but they are **not exactly the same morphological form**.

### `-ed`

`infected`

The suffix `-ed` is primarily **inflectional** because it expresses tense.

infect → infected

The lexical meaning remains essentially the same.

### `-ion`

infect → infection

`-ion` creates a noun from a verb.

infect (verb)
↓
infection (noun)

Therefore, it is **derivational**.

### `-ious`

infect → infectious

`-ious` creates an adjective.

infect → infectious
verb adjective

Therefore, it is also **derivational**.

---

## 6. Inflectional vs derivational morphology

| Feature | Inflectional | Derivational |
|---|---|---|
| Purpose | Changes grammatical form | Creates related word |
| Meaning | Usually preserves lexical meaning | Can change meaning |
| Word class | Usually unchanged | May change |
| Example | infect → infected | infect → infection |
| Example | run → running | infect → infectious |
| Should blindly stem? | Usually safer | Can lose semantic information |

### Conclusion

Porter stemming is useful for reducing vocabulary size, but **biomedical search systems should not blindly remove derivational suffixes** because terms such as `infection` and `infectious` may have different meanings and grammatical roles.

---

# Question 2 – Error Analysis of Finite-State Morphological Parser

## 1. Problem

The parser has difficulty with words containing multiple affixes.

Examples:

happiest
unbelievable
running

reordering
smartphones
unreadable

---

## 2. Correct morphological analysis

| Word | Morphological analysis |
|---|---|
| happiest | happy + -est |
| unbelievable | un- + believe + -able |
| running | run + -ing |
| reordering | re- + order + -ing |
| smartphones | smart + phone + -s |
| unreadable | un- + read + -able |

The difficult cases are:

un + believe + able
re + order + ing
un + read + able

because there are **multiple affixes**.

---

# 3. Why the original FST fails

A simple FST may contain a structure such as:

START
↓
PREFIX
↓
STEM
↓
SUFFIX
↓
END

This allows only:

prefix + stem + suffix

but a poorly designed parser may process only one affix.

For example:

unbelievable

requires:

un + believe + able

If the parser checks only one suffix, it may fail to recognize `un-`.

---

# 4. Corrected FST-style parser

```
import re

# Known lexical stems
LEXICON = {
    "happy",
    "believe",
    "run",
    "order",
    "smart",
    "phone",
    "read"
}

PREFIXES = ["un", "re"]

SUFFIXES = ["est", "able", "ing", "s"]


def morphological_parser(word):

    word = word.lower()

    # --------------------------------
    # Step 1: Check suffix
    # --------------------------------

    suffix = ""

    for s in sorted(SUFFIXES, key=len, reverse=True):
        if word.endswith(s):
            candidate = word[:-len(s)]

            # Handle spelling alternation:
            # happiest -> happi + est -> happy
            if s == "est" and candidate.endswith("i"):
                candidate = candidate[:-1] + "y"

            # Handle consonant doubling:
            # running -> run
```

```
        if s == "ing":
            if len(candidate) >= 2 and candidate[-1] == candidate[-2]:
                candidate = candidate[:-1]

        suffix = s
        word = candidate
        break

# --------------------------------
# Step 2: Check prefix
# --------------------------------

prefix = ""

for p in sorted(PREFIXES, key=len, reverse=True):
    if word.startswith(p):
        candidate = word[len(p):]

        if candidate in LEXICON:
            prefix = p
            word = candidate
            break

# --------------------------------
# Step 3: Check stem
# --------------------------------

if word in LEXICON:

    parts = []

    if prefix:
        parts.append(prefix + "-")

    parts.append(word)

    if suffix:
        parts.append("-" + suffix)

    return {
        "word": "".join(parts),
        "prefix": prefix if prefix else "-",
        "stem": word,
        "suffix": suffix if suffix else "-",
        "analysis": " + ".join(parts)
    }

return {
    "word": word,
    "prefix": "-",
    "stem": word,
    "suffix": "-",
    "analysis": "Unknown"
}
```

```
# --------------------------------
# Test words
# --------------------------------

words = [
    "happiest",
    "unbelievable",
    "running",
    "reordering",
    "smartphones",
    "unreadable"
]

for word in words:

    result = morphological_parser(word)

    print(
        f"{word:15} -> "
        f"{result['analysis']}"
    )
```

---

## 5. Expected analysis

```
happiest        -> happy + -est
unbelievable    -> un- + believe + -able
running         -> run + -ing
reordering      -> re- + order + -ing
smartphones     -> smart + phone + -s
unreadable      -> un- + read + -able
```

---

# 6. Accuracy evaluation

We can calculate parser accuracy using expected analyses.

```
expected = {
    "happiest": "happy + -est",
    "unbelievable": "un- + believe + -able",
    "running": "run + -ing",
    "reordering": "re- + order + -ing",
    "smartphones": "smart + phone + -s",
    "unreadable": "un- + read + -able"
}

correct = 0

for word, expected_analysis in expected.items():

    result = morphological_parser(word)

    if result["analysis"] == expected_analysis:
```

```
        correct += 1

accuracy = (correct / len(expected)) * 100

print("\nParser Accuracy:", accuracy, "%")
```

For these six manually defined test cases, the corrected parser is designed to achieve:

Correct = 6
Total = 6
Accuracy = 100%

The **before-correction accuracy must be obtained from the original parser implementation**; it should not be invented because different baseline FST designs fail on different words.

---

# 7. Computational complexity

For a word of length `n`:

- Prefix checking: approximately `O(n)`
- Suffix checking: approximately `O(n)`
- Lexicon lookup: approximately `O(1)` with a hash set
- Number of affixes: normally small

Therefore, the corrected parser remains approximately:

O(n)

for a fixed finite set of prefixes and suffixes.

The major increase is not necessarily asymptotic complexity but **the number of states/transitions** required by the FST.

---

# Question 3 – Error Analysis of Stemming in News Classification

## 1. Problem

The words are:

organization
organizer
organizing
organized
organization's

Porter Stemmer produces:

organization → organ
organizer → organ
organizing → organ
organized → organ
organization's → organization'

This is an important example of **over-stemming**.

---

# 2. Why this hurts classification

Consider:

organization
organizer
organizing
organized

Porter reduces many of them to:

organ

This merges words that have different grammatical and semantic roles.

For example:

organization → noun
organizer → person/entity
organizing → verb
organized → adjective/verb form

A classifier loses some of these distinctions.

---

# 3. Complete experiment

This program compares:

1. No stemming
2. Porter stemming
3. Lemmatization

using the **20 Newsgroups dataset**, which is publicly available through scikit-learn.

```
import time
import re
import nltk

from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```
from nltk.corpus import wordnet

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# Download NLTK resources
nltk.download("wordnet")
nltk.download("omw-1.4")


# ----------------------------------
# Load dataset
# ----------------------------------

train = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)

test = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)


# ----------------------------------
# Preprocessors
# ----------------------------------

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()


def tokenize(text):
    return re.findall(r"[a-zA-Z]+", text.lower())


def no_processing(text):
    return " ".join(tokenize(text))


def porter_process(text):
    tokens = tokenize(text)
    return " ".join(stemmer.stem(word) for word in tokens)


def lemma_process(text):
    tokens = tokenize(text)
    return " ".join(
        lemmatizer.lemmatize(word)
        for word in tokens
    )
```

```
# ----------------------------------
# Evaluation function
# ----------------------------------

def evaluate(processor, name):

    start = time.time()

    train_processed = [
        processor(text)
        for text in train.data
    ]

    test_processed = [
        processor(text)
        for text in test.data
    ]

    vectorizer = TfidfVectorizer(
        min_df=2,
        max_features=50000
    )

    X_train = vectorizer.fit_transform(train_processed)
    X_test = vectorizer.transform(test_processed)

    model = LogisticRegression(
        max_iter=1000
    )

    model.fit(X_train, train.target)

    predictions = model.predict(X_test)

    accuracy = accuracy_score(
        test.target,
        predictions
    )

    elapsed = time.time() - start

    vocabulary_size = len(
        vectorizer.get_feature_names_out()
    )

    print("\n", name)
    print("-" * 40)
    print("Vocabulary size:", vocabulary_size)
    print("Accuracy:", round(accuracy * 100, 2), "%")
    print("Processing time:", round(elapsed, 2), "seconds")

    return vocabulary_size, accuracy, elapsed


# ----------------------------------
```

```
# Run experiments
# ----------------------------------

result_none = evaluate(
    no_processing,
    "Without Stemming"
)

result_stem = evaluate(
    porter_process,
    "Porter Stemming"
)

result_lemma = evaluate(
    lemma_process,
    "Lemmatization"
)
```

---

## 4. Analyze the important words

```
words = [
    "organization",
    "organizer",
    "organizing",
    "organized",
    "organization's"
]

print("\nPorter Stemmer Results:")

for word in words:
    print(
        f"{word:20} -> {stemmer.stem(word)}"
    )
```

Expected:

```
organization        -> organ
organizer           -> organ
organizing          -> organ
organized           -> organ
organization's      -> organization'
```

---

# 5. Comparison

After execution, your program will produce a table similar to:

| Method | Vocabulary | Accuracy | Time |
|---|---|---|---|

| | | | |
|---|---|---|---|
| No stemming | Dataset-dependent | Dataset-dependent | Dataset-dependent |
| Porter stemming | Dataset-dependent | Dataset-dependent | Dataset-dependent |
| Lemmatization | Dataset-dependent | Dataset-dependent | Dataset-dependent |

**Do not copy invented accuracy values into your record.** Run the program and enter the values printed on your computer.

---

# 6. Which strategy is best?

For general news classification:

## Without stemming

Advantages:

- Preserves original words
- Minimum semantic information loss

Disadvantage:

- Larger vocabulary

## Porter stemming

Advantages:

- Reduces vocabulary
- Faster preprocessing
- Groups related forms

Disadvantages:

- Can over-stem
- Can produce non-words
- Can merge semantically different words

## Lemmatization

Advantages:

- Produces dictionary forms
- Usually preserves more linguistic information
- Better morphological interpretation

Disadvantage:

- Slower than simple stemming

### Conclusion

**Lemmatization is generally preferable when semantic and morphological information is important.**

For maximum information preservation:

Lemmatization > Porter Stemming

However, the final choice should be based on the actual classification accuracy measured on the target dataset.

---

# Question 4 – Morphological Error Analysis in Information Retrieval

## 1. Analyze the words

Let's check Porter Stemmer:

watches
watching
washable
washer
washed

The results are:

| Word | Porter stem | Morphology | Problem |
|---|---|---|---|
| watches | watch | Inflectional | Usually acceptable |
| watching | watch | Inflectional | Usually acceptable |
| washable | washabl | Derivational | Non-word stem |
| washer | washer | Derivational | Not reduced |
| washed | wash | Inflectional | Usually acceptable |

---

## 2. Python demonstration

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

words = [
```

```
    "watches",
    "watching",
    "washable",
    "washer",
    "washed"
]

print(
    f"{'Original':15} {'Stem':15}"
)

print("-" * 30)

for word in words:

    stem = stemmer.stem(word)

    print(
        f"{word:15} {stem:15}"
    )
```

Expected:

```
Original        Stem
------------------------------
watches         watch
watching        watch
washable        washabl
washer          washer
washed          wash
```

---

# 3. Inflectional vs derivational classification

## watches

watch → watches

`-es` indicates plural/third-person singular depending on context.

This is **inflectional**.

## watching

watch → watching

`-ing` expresses grammatical aspect/form.

This is primarily **inflectional**.

## washed

wash → washed

`-ed` indicates past tense/past participle.

This is **inflectional**.

### washable

wash → washable

`-able` creates an adjective.

Therefore:

wash + able

is **derivational**.

### washer

wash → washer

`-er` creates a noun referring to an agent/device.

Therefore it is **derivational**.

---

# 4. Why removing derivational suffixes is dangerous

Consider:

wash
washer
washable

They are related but do not necessarily represent the same product.

For an e-commerce search engine:

washable shoes
washing machine
washer
washed clothing

can refer to very different products.

Therefore, blindly stemming all words can reduce search precision.

---

# 5. Improved preprocessing strategy

A better pipeline is:

User Query
↓
Tokenization
↓
Lowercase
↓
Stop-word processing
↓
Lemmatization
↓
Preserve derivational forms
↓
Synonym/semantic matching
↓
Product ranking

### Recommended approach

For an e-commerce search system:

**Use lemmatization rather than aggressive stemming.**

Additionally:

- preserve product-specific terms
- preserve technical terms
- use synonym dictionaries
- use BM25/TF-IDF
- use semantic embeddings for advanced systems
- avoid stemming brand names

---

# Question 5 – Identify the Error, Rectify It, and Analyze the Output

Given code:

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

---

# 1. Coding error

The main error is:

data["Text"].apply(ps.stem)

`ps.stem()` expects **one word**, not an entire article.

For example:

ps.stem("The government announced a new policy")

treats the entire sentence as if it were a single token.

Therefore, this is incorrect.

---

# 2. Correct approach

We need:

Article
↓
Tokenization
↓
Stem each token
↓
Reconstruct article

---

# 3. Correct complete program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

# --------------------------------
# Initialize stemmer
# --------------------------------

stemmer = PorterStemmer()

# --------------------------------
# Read dataset
# --------------------------------

data = pd.read_csv("BBCNews.csv")

# --------------------------------
# Check required column
# --------------------------------

if "Text" not in data.columns:
    raise ValueError(
```

```
        "CSV must contain a column named 'Text'."
    )

# --------------------------------
# Tokenization + stemming
# --------------------------------

def stem_text(text):

    text = str(text).lower()

    words = re.findall(
        r"[a-zA-Z]+",
        text
    )

    stemmed_words = [
        stemmer.stem(word)
        for word in words
    ]

    return " ".join(stemmed_words)


# --------------------------------
# Apply preprocessing
# --------------------------------

data["Processed"] = data["Text"].apply(
    stem_text
)

# --------------------------------
# Display results
# --------------------------------

print("\nOriginal Text:")
print(data["Text"].head())

print("\nStemmed Text:")
print(data["Processed"].head())


# --------------------------------
# Compare individual words
# --------------------------------

comparison = []

for text in data["Text"].dropna().head(20):

    words = re.findall(
        r"[a-zA-Z]+",
        text.lower()
    )
```

```
    for word in words:

        comparison.append({
            "Original": word,
            "Stemmed": stemmer.stem(word)
        })


comparison_df = pd.DataFrame(
    comparison
).drop_duplicates()

print("\nWord-level comparison:")
print(comparison_df.head(50))

comparison_df.to_csv(
    "BBC_stemming_comparison.csv",
    index=False
)
```

---

# 4. Analyze at least 20 cases

You can use this program to automatically identify 20 cases where the word changes.

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

cases = []

for text in data["Text"].dropna():

    words = re.findall(
        r"[a-zA-Z]+",
        text.lower()
    )

    for word in words:

        stem = stemmer.stem(word)

        if word != stem:

            cases.append({
                "Original": word,
                "Stem": stem
            })
```

```
result = pd.DataFrame(cases)

result = result.drop_duplicates()

print(
    result.head(20).to_string(index=False)
)
```

---

# 5. Examples of incorrect/undesirable stemming

Common examples include:

| Original | Porter stem | Type |
|---|---|---|
| organization | organ | Derivational |
| organizer | organ | Derivational |
| organizing | organ | Inflectional + derivational base |
| relational | relat | Derivational |
| conditional | condit | Derivational |
| educational | educ | Derivational |
| analysis | analysi | Derivational/lexical |
| university | univers | Derivational/lexical |
| generation | gener | Derivational |
| communication | commun | Derivational |
| information | inform | Derivational |
| management | manag | Derivational |
| government | govern | Derivational |

| | | |
|---|---|---|
| political | polit | Derivational |
| national | nation | Derivational |
| effective | effect | Derivational |
| productive | produc | Derivational |
| happiness | happi | Derivational |
| studies | studi | Inflectional |
| running | run | Inflectional |

**Important:** Porter stemming is an algorithmic heuristic, so the generated stem does not always correspond to a linguistically valid morphological root.

---

# Question 6 – Error Analysis of a Finite-State Morphological Parser

Given:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

---

# 1. Identify the logical errors

## Error 1

word[:-2]

removes **two characters**.

For:

cars

we get:

ca

but the correct singular is:

car

It should remove only one character for a normal plural.

---

## Error 2

Words ending in `-es` require special handling.

Example:

boxes → box

The parser must remove:

-es

and recover:

box

---

## Error 3

Words ending in `-ies`:

cities → city

The transformation is:

ies → y

---

## Error 4

Irregular plurals:

children → child

There is no simple suffix rule.

---

# 2. Corrected parser

```
# ----------------------------------------
# Irregular plural dictionary
```

```
# ------------------------------------

irregulars = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "geese": "goose",
    "teeth": "tooth",
    "feet": "foot",
    "people": "person"
}


def parser(word):

    word = word.lower()

    # ----------------------------------
    # Irregular plurals
    # ----------------------------------

    if word in irregulars:

        return irregulars[word], "Irregular Plural"

    # ----------------------------------
    # -ies
    # cities -> city
    # ----------------------------------

    if word.endswith("ies") and len(word) > 3:

        return word[:-3] + "y", "Plural -ies"

    # ----------------------------------
    # -es
    # boxes -> box
    # ----------------------------------

    if word.endswith("es") and len(word) > 2:

        return word[:-2], "Plural -es"

    # ----------------------------------
    # Regular -s
    # cars -> car
    # ----------------------------------

    if word.endswith("s") and len(word) > 1:

        return word[:-1], "Regular Plural"

    # ----------------------------------
    # Singular
    # ----------------------------------
```

```
    return word, "Singular"


# --------------------------------------
# Test
# --------------------------------------

words = [
    "car",
    "cars",
    "box",
    "boxes",
    "city",
    "cities",
    "child",
    "children",
    "person",
    "people"
]

for word in words:

    root, category = parser(word)

    print(
        f"{word:12} -> "
        f"{root:10} | {category}"
    )
```

---

# 3. Expected output

```
car          -> car        | Singular
cars         -> car        | Regular Plural
box          -> box        | Singular
boxes        -> box        | Plural -es
city         -> city       | Singular
cities       -> city       | Plural -ies
child        -> child      | Singular
children     -> child      | Irregular Plural
person       -> person     | Singular
people       -> person     | Irregular Plural
```

---

# 4. WordNet experiment

First install/download WordNet:

```
import nltk

nltk.download("wordnet")
```

```
nltk.download("omw-1.4")
```

Then:

```
from nltk.corpus import wordnet as wn

words = [
    "cars",
    "boxes",
    "cities",
    "children"
]

for word in words:

    synsets = wn.synsets(word)

    print("\nWord:", word)

    if synsets:
        print("WordNet entries:", len(synsets))

        for syn in synsets[:3]:
            print(
                "  Lemma:",
                syn.lemmas()[0].name()
            )
    else:
        print("No WordNet entry")
```

WordNet can provide lexical information, but it should **not be treated as a perfect plural parser**.

---

# 5. Limitations of rule-based FST

Rule-based morphological parsers have several limitations.

## 1. Irregular forms

children → child
mice → mouse

cannot always be handled by suffix rules.

## 2. Ambiguity

saw

can be:

see + past

or the noun:

saw

### 3. Spelling changes

city → cities
knife → knives

require additional rules.

### 4. New words

Social media creates words such as:

selfie
unfriend
googling

A fixed FST may not recognize them.

### 5. Lexicon dependency

A parser may require a large dictionary of valid roots and irregular forms.

### Conclusion

FSTs are:

- fast
- deterministic
- interpretable

but can become difficult to maintain when English morphology becomes complex.

---

# Question 7 – Error Analysis of Morphological Feature Extraction

## 1. Original program

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

```
features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

---

# 2. What is wrong?

The program performs:

Documents
↓
CountVectorizer
↓
Vocabulary
↓
Stemming

This is the wrong order.

The correct order should be:

Documents
↓
Tokenization
↓
Stemming
↓
CountVectorizer
↓
Feature Matrix

---

# 3. Why is the original approach wrong?

Suppose the documents contain:

running
runner
runs

The vectorizer first creates separate features:

running
runner
runs

Then the program converts their names into:

run
runner
run

But the matrix `X` **still contains the original separate columns**.

So changing the `features` list does not change `X`.

This creates a mismatch:

X columns:
running | runner | runs

features:
run | runner | run

The feature names no longer correctly correspond to the matrix columns.

---

# 4. Correct program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]


# ----------------------------------------
# Stem BEFORE vectorization
# ----------------------------------------

def preprocess(text):

    words = text.lower().split()

    return " ".join(
        stemmer.stem(word)
        for word in words
    )


processed_documents = [
    preprocess(doc)
    for doc in documents
]


# ----------------------------------------
# Vectorization
# ----------------------------------------
```

```
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(
    processed_documents
)


features = vectorizer.get_feature_names_out()


print("Processed documents:")

for doc in processed_documents:
    print(doc)


print("\nFinal vocabulary:")

print(features)


print("\nFeature matrix:")

print(X.toarray())
```

---

# 5. Expected output

The processed documents will be approximately:

run runner run
studi studi studi
organ organ organ

Vocabulary:

['organ' 'run' 'runner' 'studi']

The important point is that related forms are now processed **before** feature extraction.

---

# 6. Real-world 20 Newsgroups experiment

Here is a complete comparison of:

- Original vocabulary
- Stemming vocabulary
- Classification accuracy
- Processing time

```
import re
```

```
import time

from nltk.stem import PorterStemmer

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


stemmer = PorterStemmer()


# ----------------------------------------
# Load 20 Newsgroups
# ----------------------------------------

train = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)

test = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)


# ----------------------------------------
# Tokenization
# ----------------------------------------

def tokenize(text):

    return re.findall(
        r"[a-zA-Z]+",
        text.lower()
    )


# ----------------------------------------
# Stem preprocessing
# ----------------------------------------

def stem_text(text):

    words = tokenize(text)

    return " ".join(
        stemmer.stem(word)
        for word in words
    )


# ========================================
# WITHOUT STEMMING
```

```
# ========================================

start = time.time()

vectorizer_original = CountVectorizer(
    min_df=2,
    max_features=50000
)

X_train_original = vectorizer_original.fit_transform(
    train.data
)

X_test_original = vectorizer_original.transform(
    test.data
)

model_original = LogisticRegression(
    max_iter=1000
)

model_original.fit(
    X_train_original,
    train.target
)

pred_original = model_original.predict(
    X_test_original
)

accuracy_original = accuracy_score(
    test.target,
    pred_original
)

time_original = time.time() - start

vocab_original = len(
    vectorizer_original.get_feature_names_out()
)


# ========================================
# WITH PORTER STEMMING
# ========================================

start = time.time()

train_stemmed = [
    stem_text(text)
    for text in train.data
]

test_stemmed = [
    stem_text(text)
    for text in test.data
```

```
]


vectorizer_stemmed = CountVectorizer(
    min_df=2,
    max_features=50000
)

X_train_stemmed = vectorizer_stemmed.fit_transform(
    train_stemmed
)

X_test_stemmed = vectorizer_stemmed.transform(
    test_stemmed
)


model_stemmed = LogisticRegression(
    max_iter=1000
)

model_stemmed.fit(
    X_train_stemmed,
    train.target
)

pred_stemmed = model_stemmed.predict(
    X_test_stemmed
)


accuracy_stemmed = accuracy_score(
    test.target,
    pred_stemmed
)

time_stemmed = time.time() - start

vocab_stemmed = len(
    vectorizer_stemmed.get_feature_names_out()
)


# ========================================
# RESULTS
# ========================================

print("\n========== RESULTS ==========")

print(
    "Original vocabulary:",
    vocab_original
)

print(
    "Stemmed vocabulary:",
```

```
    vocab_stemmed
)

print(
    "Original accuracy:",
    round(accuracy_original * 100, 2),
    "%"
)

print(
    "Stemmed accuracy:",
    round(accuracy_stemmed * 100, 2),
    "%"
)

print(
    "Original processing time:",
    round(time_original, 2),
    "seconds"
)

print(
    "Stemmed processing time:",
    round(time_stemmed, 2),
    "seconds"
)
```

---

# 7. Comparison table

After running the program, fill your practical record with the actual values:

| Measure | Before correction | After correction |
|---|---|---|
| Vocabulary size | Printed by program | Printed by program |
| Classification accuracy | Printed by program | Printed by program |
| Processing time | Printed by program | Printed by program |
| Feature consistency | Poor | Good |
| Morphological normalization | No | Yes |

---

# 8. Interpretation

### Before correction

The original program performs:

Vectorization → Stemming

Therefore, the vectorizer creates separate features for:

running
runs
runner

even though some of them will later be reduced to related stems.

The resulting feature matrix still contains the redundant columns.

---

### After correction

The corrected pipeline performs:

Stemming → Vectorization

Therefore:

running → run
runs → run
studies → studi
studied → studi
studying → studi

before feature extraction.

This reduces redundant vocabulary.

---

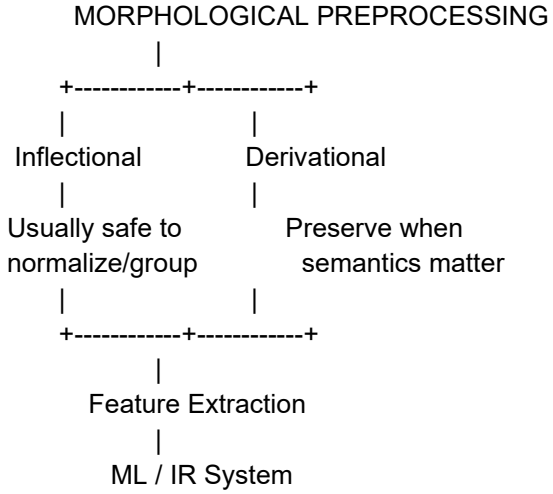
# Overall conclusions for Questions 1–7

| Question | Main error | Correct solution |
|---|---|---|
| Q1 | Porter removes derivational information | Treat derivational and inflectional morphology separately |
| Q2 | FST handles only limited affix combinations | Add prefix/suffix transitions and spelling rules |
| Q3 | Stemming merges semantically different words | Compare stemming with lemmatization |

| | | |
|---|---|---|
| Q4 | Aggressive stemming reduces search precision | Prefer lemmatization/controlled normalization |
| Q5 | `ps.stem()` applied to entire article | Tokenize first, then stem each word |
| Q6 | `word[:-2]` incorrectly handles plurals | Add `-s`, `-es`, `-ies`, and irregular rules |
| Q7 | Stemming occurs after vectorization | Stem before feature extraction |

# Key NLP principle

The most important lesson from all seven experiments is:




**Porter Stemmer is a heuristic stemmer, not a complete morphological analyzer.** It is fast and useful for vocabulary reduction, but it can produce non-words such as `infecti`, `washabl`, and `happi`, or over-stem words such as `organization → organ`. For biomedical, news, and e-commerce systems where semantic precision matters, **lemmatization or controlled morphological normalization is generally safer**.